Avaliação de Desempenho entre Banco de Dados NoSQL e Modelo Relacional

CAUAN JOSÉ DA SILVA GUERRA

Orientadora: Bruna Vaz Vieira Brandão

ITABERAÍ

2021

CAUAN JOSÉ DA SILVA GUERRA

Avaliação de Desempenho entre Banco de Dados NoSQL e Modelo Relacional

Monografia apresentada para obtenção do título de Bacharel à banca examinadora do Curso de Bacharelado em Sistemas de Informação da Universidade Estadual de Goiás, Unidade de Itaberaí. Orientadora: Bruna Vaz Vieira Brandão.

ITABERAÍ - GO

2021

Elaborada pelo Sistema de Geração Automática de Ficha Catalográfica da UEG
com os dados fornecidos pelo(a) autor(a).

JG934 José da Silva Guerra, Cauan
a Avaliação de Desempenho entre Banco de Dados NoSQL e Modelo Relacional / Cauan José da Silva Guerra; orientador Bruna Vaz Vieira Brandão. -- Itaberaí, 2022.
47 p.

Graduação - Sistemas de Informação -- Unidade de Itaberaí, Universidade Estadual de Goiás, 2022.

1. Banco de dados. 2. Modelo relacional. 3. NoSQL. I. Vieira Brandão, Bruna Vaz , orient. II. Título.

SERVIÇO PÚBLICO ESTADUAL
UNIVERSIDADE ESTADUAL DE GOIÁS
UNIDADE UNIVERSITÁRIA DE ITABERAÍ
Rua 05, Qd. A, - Jardim Cabral – CEP 76.630-000 – Fax: (062) 3375-4876 / Fone:3375-4877 – Itaberaí – Goiás.

ANEXO IV

TERMO DE AUTORIZAÇÃO PARA DISPONIBILIZAÇÃO DE TRABALHOS ACADÊMICOS ELETRÔNICOS NO BANCO DE MONOGRAFIAS DA UNIDADE UNIVERSITÁRIA DE ITABERAÍ

Eu, **Cauan José da Silva Guerra** , portador do RG n° **6216820**_ Org. Exp.**SSP-GO**, inscrito no CPF sob nº **70289823196**, domiciliado no logradouro de nome Rua Alameda Jambeiro, na cidade de Itaberaí, estado de Goiás.

Na qualidade de titular dos direitos de autor que recaem sobre a minha monografia de conclusão de curso, intitulada **Avaliação de desempenho entre Banco de Dados NoSQL e modelo Relacional**, defendida em **17/03/2022**, junto a banca examinadora do curso com fundamento nas disposições da lei nº 9.610 de 19 de fevereiro de 1998, autorizo ou não a disponibilizar gratuitamente a obra citada, sem ressarcimento de direitos autorais, para fins de leitura, impressão e/ou *downloading* pela *internet*, a título de divulgação da produção científica gerada pela Universidade Estadual de Goiás / UnU de Itaberaí , a partirdesta data.

☒ autorizo ☐ não autorizo

☐ total ☐ resumo

Assim, **autorizo ou não a liberação total/resumo de meu trabalho, estando ciente que o conteúdo disponibilizado é de minha inteira responsabilidade.**

Itaberaí, 17 de 03 de 2022.

Cauan José da Silva Guerra

Assinatura(s) do(s) autor(s).

Ata de Qualificação do Trabalho de Curso I

Às 20 horas e 50 minutos do dia 17 de Março de 2022 em seção remota mediada pelo *Google Meet*, procedeu-se a qualificação do Trabalho de Curso de Bacharelado em Sistemas de Informação, o/a acadêmica: **Cauan José da Silva Guerra** com o tema: DESENVOLVIMENTO DE APLICATIVO PARA MONITORAMENTO DE EXERCÍCIOS FÍSICOS sob as orientações da professora: **Bruna Vaz Vieira Brandão**

À oportunidade foi convidado como leitor, a professora: **Rogéria Luzia Wolpp Gonçalves** e o professor **Odiney Candido Jacomini** para fazer parte da Banca Examinadora. Após realizada a apresentação da acadêmica no período mínimo 10 (dez) e máximo 15 (quinze) minutos, foi aberto espaço para as arguições do(a) professor(a) convidado(a). Em seguida, o professor(a) orientador(a), responsável por presidir a Banca Examinadora, reuniu-se com o(a) professor(a) avaliador(a) para atribuir a nota final de **9,5 pontos**, estando o acadêmica **Cauan José da Silva Guerra.**

[X] APROVADO [ ] REPROVADO ou [ ] EXAME FINAL

na disciplina Trabalho de Curso I que representa o pré-requisito para a matrícula na disciplina Trabalho de Curso II e a continuidade na execução do Trabalho de Curso. A banca recomenda a regularização às seguintes ressalvas:

Sendo esta a expressão da verdade, eu professora **Bruna Vaz Vieira Brandão**, lavrei a presente Ata, que após lida e achada conforme vai ser assinada por todos e entregue à Secretaria da Unidade Universitária de Itaberaí, para os devidos fins.

Odiney Candido Jacomini
Professor Leitor

Rogéria Luzia Wolpp Gonçalves
Professora Leitora

Bruna Vaz Vieira Brandão
Professora Orientadora

Este trabalho é dedicado aos meus colegas

e em especial a minha família.

AGRADECIMENTOS

A Deus, pois sem ele nada disso seria possível;

A minha mãe, por sempre estar ao meu lado me incentivando e dando apoio;

A minha orientadora, Bruna Vaz Vieira Brandão, pela paciência e atenção dedicada;

E muito obrigado a todos que contribuíram de forma direta ou indiretamente para a conclusão do trabalho.

“Viva com orgulho, se sua fraqueza o dominar, aqueça o seu coração, cerre os dentes e siga em frente. Mesmo que sua covardia o retarde, não impedirá a passagem do tempo ” (Kyojuro Rengoku | Kimetsu no Yaiba)

RESUMO

A evolução das aplicações trouxe um grande aumento do volume de dados, esse aumento criou uma necessidade de armazenar dados em grande escala de forma segura, que está cada vez mais importante. É de extrema importância possuir uma estrutura flexível que irá suprir todas as necessidades, portanto, é natural o uso de NoSQL nesses tipos de aplicações. Com o intuito de melhorar o desempenho de suas aplicações, muitos desenvolvedores já estão fazendo uso deste paradigma que tem como propósito uma maior flexibilidade. Entretanto, existem muitos questionamentos sobre as vantagens e desvantagens do uso do paradigma não relacional em comparação ao paradigma relacional. Desta forma, este trabalho tem por objetivo apresentar um estudo comparativo de desempenho entre bancos de dados NoSQL e banco de dados relacional. Todos os testes a serem realizados usará operações básicas e fundamentais do SQL de INSERT, DELETE, SELECT e UPDATE e será medido quantitativamente o tempo de execução de cada operação. Cada teste será representado em forma de gráficos ou tabelas, para serem analisados e discutidos posteriormente.

Palavras-chave: Banco de dados. NoSQL.

ABSTRACT

The evolution of applications has brought a huge increase in data volume, this increase has created a need to store large-scale data securely, which is increasingly important. It is extremely important to have a flexible structure that will meet all needs, so it is natural to use NoSQL in these types of applications. In order to improve the performance of their applications, many developers are already making use of this paradigm, which aims to provide greater flexibility. However, there are many questions about the advantages and disadvantages of using the non-relational paradigm compared to the relational paradigm. Thus, this work aims to present a comparative study of performance between NoSQL databases and relational databases. All tests to be performed will use basic and fundamental SQL operations of INSERT, DELETE, SELECT and UPDATE and the execution time of each operation will be measured quantitatively. Each test will be represented in the form of graphs or tables, to be analyzed and discussed later.

Keywords: Database. NoSQL.

LISTA DE FIGURAS

Figura 1 - Escalonamento Horizontal ……..…….23
Figura 2 - Replicação de Dados ……..……..……...……24
Figura 3 - Exemplo de documento ……..…...……..26
Figura 4 - Dados armazenados em chave-valor ……....……..27
Figura 5 - Exemplo de modelo de dados baseado em colunas …….……..28
Figura 6 - Estrutura baseada em grafos ……..……29
Figura 7 - Modelo entidade e relacionamento …….…….……..37
Figura 8 - Modelo orientado a documento ……..…….……38
Figura 9 - Gráfico Insert …….……...……39
Figura 10 - Gráfico Delete ……..……..……..40
Figura 11 - Gráfico Update ……..……..……41
Figura 12 - Gráfico Select …….……...……...……..42

LISTA DE TABELAS E QUADROS

Tabela 1 - Características SGBDs e casos de sucesso ……30
Tabela 2 - Análise comparativa Modelo Relacional e NoSQL ……..32
Tabela 3 - Configurações do computador ……35
Tabela 4 - Softwares utilizados ……..43
Quadro 1 - Insert MySQL realizado no NetBeans ……..39
Quadro 2 - Delete MySQL realizado no NetBeans ……40
Quadro 3 - Update MySQL realizado no NetBeans ……41
Quadro 4 - Select MySQL realizado no NetBeans ……..42

LISTA DE ABREVIATURAS E SIGLAS

SGDB - Sistema de Gerenciamento de Banco de Dados

SQL - Structured Query Language

BD - Banco de Dados

MR - Modelo Relacional

NOSQL - Not Only SQL

ACID - Atomicidade, Consistência, Isolamento, Durabilidade

XML - Extensible Markup Language

JSON - JavaScript Object Notation

API - Applications protocol interface

MRN - Modelo relacional normalizado

SUMÁRIO

1. Introdução ……...15
1.1.Objetivo Geral…….……16
1.2.Objetivos Específicos…….……16
1.3.Estrutura do Trabalho…….……16
2. Fundamentação Teórica…….……..17
2.1. Modelo Relacional…….……..……17
2.1.1. Características……..18
2.1.2. Operações……..18
2.1.3. ACID (Atomicidade, Consistência, Isolamento, Durabilidade)……19
2.1.4. Normalização ……19
2.1.5. Limitações ……..20
2.2. Banco de dados NoSQL ……..20
2.2.1. Principais características dos bancos de dados NoSQL ……22
2.2.1.1. Escalabilidade horizontal…….……22
2.2.1.2. Ausência de esquema ou esquema flexível…….……23
2.2.1.3. Suporte nativo a replicação……23
2.2.1.4. API simples para acesso aos dados ……..24
2.2.2. Tipos de Modelo de Dados no NoSQL ……..24
2.2.2.1. Sistemas orientados a Documentos ……..25
2.2.2.2. Sistema baseado em Chave-valor ……26
2.2.2.3. Sistemas orientados a Colunas ……..28
2.2.2.4. Sistemas orientados a Grafos ……29
2.2.2.4 Sistemas de Gerenciamento de Banco de Dados NoSQL ……..30
2.3. Banco de Dados Relacional x Banco de Dados NoSQL ……31
2.4. Trabalhos relacionados ……33
2.5. Considerações finais ……34
3. Desenvolvimento ……..35
3.1 Ambiente de Testes ……35
3.2 Cenário ……..36
3.3 Arquitetura do sistema ……36
3.4 Modelo relacional ……37
3.4.1 Modelo NoSQL ……...38
3.5 Realização dos testes ……38
3.5.1 INSERT ……39
3.5.2 DELETE ……...40
3.5.3 UPDATE ……..41
3.5.4 SELECT …….…..42
3.6 Softwares utilizados ..……..… 43
3.7 Análise dos dados ……...43
4. Considerações finais……..44

1. Introdução

A evolução das aplicações consequentemente trouxe um grande aumento do volume de dados e através desse aumento houve uma necessidade requisitada pelos usuários em diversas aplicações. Com isso, a necessidade de armazenar dados em grande escala de forma segura está cada vez mais importante.

Para suprir todas necessidades, os desenvolvedores de software utilizam-se da ajuda dos Bancos de Dados (BD). Entre os modelos de BD existentes, o modelo Relacional tem uma ênfase maior quando se diz respeito ao armazenamento de dados, porém não possui uma estrutura capaz de ocasionar escalabilidade suficiente para dar suporte às aplicações que têm um elevado volume de dados e consequentemente não tendo um bom desempenho.

Segundo Leite (2010, p.15 apud Brito 2010) mesmo com os recursos disponíveis pelo uso de SGBDs relacionais, em cenários onde se concentra um grande volume de dados, o modelo relacional já não é eficiente pelo fato de que quanto mais volume de dados é necessário mais escalabilidade, resultando em um modelo não muito eficiente.

Com base nas afirmações anteriores, é relevante a importância de um modelo de banco de dados (BD) capaz de trabalhar com um fluxo de dados mais elevado. Uma opção para resolver essa questão é a utilização do NoSQL (Not Only SQL).

Com o intuito de melhorar o desempenho de suas aplicações, muitos desenvolvedores já estão fazendo uso deste paradigma que tem como propósito uma maior flexibilidade.

Entretanto, ainda existem muitos questionamentos a respeito do modelo NoSQL, um dos principais questionamento é quando se diz a respeito do seu uso e o seu desempenho tendo como base o tempo de execução das operações. Muitas discussões são levantadas em relação ao tempo de execução, fazendo a comparação entre o modelo relacional (MR) e o NoSQL (Not only SQL). Desta forma, este trabalho teve por objetivo apresentar um estudo comparativo de

desempenho entre bancos de dados NoSQL e o banco de dados relacional, facilitando na escolha do modelo de banco de dados que irá utilizar.

1.1.Objetivo Geral

Elaborar uma comparação de desempenho tendo como objetivo analisar o tempo de execução das instruções INSERT, DELETE, SELECT e UPDATE dos dados armazenados utilizando duas tecnologias distintas : banco de dados NoSQL e o banco de dados relacional.

1.2.Objetivos Específicos

- Abordar as principais características do modelo NoSQL e Relacional.
- Criar um cenário controlado de testes para poder avaliar o desempenho do banco de dados NoSQL e Relacional, tendo como base o tempo de execução.
- Apresentar os dados obtidos.

1.3.Estrutura do Trabalho

O trabalho apresentado está dividido em 3 capítulos, no primeiro capítulo foi apresentado uma breve introdução e qual é o objetivo do trabalho.
No segundo capítulo, é abordado todas as características do banco de dados NoSQL e do modelo Relacional.

No terceiro e último capítulo, foi desenvolvido um estudo de caso, em que foi definido o cenário para poder executar todos os testes e avaliar os resultados obtidos pelo o mesmo.

2. Fundamentação Teórica

Neste capítulo serão abordadas as principais características do banco de dados modelo relacional e do modelo NoSQL.

2.1. Modelo Relacional

Com o seu surgimento em meados dos anos 70, o modelo relacional (MR) foi muito importante para a área de banco de dados, tornou-se um modelo muito bem aceito e desde então tem sido a principal solução mesmo com o surgimento de novos softwares.

> "Os primeiros sistemas relacionais experimentais desenvolveram-se no fim dos anos 70 e os SGBDRs (sistemas de gerenciamento de banco de dados relacional) introduzidos no início dos anos 80 eram muito lentos, pois não usavam ponteiros para o armazenamento físico ou registros de localização para acessar os registros de dados relacionados. Com o desenvolvimento de novas técnicas de armazenamento e indexação, e com o processamento aprimorado de consultas e otimização, seu desempenho melhorou. Assim, os bancos de dados relacionais tornaram-se os tipos dominantes de sistemas para as aplicações tradicionais de banco de dados. " (ELMASRI; NAVATHE, 2005, p.16) .

Segundo Elmasri (2005, p.20) o modelo relacional utiliza os conceitos de entidade, atributo e relacionamento. Uma entidade pode ser representada como um objeto do cotidiano que teria suas respectivas características sendo classificada como atributos, esses objetos podem se relacionar sendo denominado como relacionamento. Esse modelo utiliza-se de tabelas (entidades) para armazenar os dados e se relacionar com tabelas distintas. Para fazer a manipulação dos dados é

preciso utilizar-se de uma linguagem que possibilita as operações de consulta, inserção, alteração e exclusão dos dados. Pelo fato de ocorrer essa manipulação de dados é preciso possuir uma integridade dos dados, utilizando-se das regras de normalização e do paradigma ACID (Atomicidade, Consistência, Isolamento, Durabilidade).

2.1.1. Características

"O Modelo Relacional (MR) pode ser apresentado por uma ou mais tabelas que são utilizadas para representar os dados e as suas relações. Para garantir a integridade dos dados, são utilizadas chaves." (ELMASRI, 2005, p.41).

As chaves exercem a função de estabelecer o relacionamento entre as distintas tabelas. Essas chaves podem ser classificadas como chaves primária ou estrangeira, as chaves primárias são valores que nunca se repetem, desta forma pode ser usada como um índice que possibilita criar relacionamento entre as demais tabelas do seu banco de dados. Já as chaves estrangeiras é um campo que aponta para a chave primária de outra tabela, ou seja, quando existe um relacionamento entre tabelas uma tabela terá a chave estrangeira da outra , isso ocorre para garantir a integridade dos dados (ELMASRI, 2005, p.49).

2.1.2. Operações

Segundo Navathe (2006, p.100), o Modelo Relacional utiliza-se da linguagem SQL (Structured Query Language), essa linguagem possibilita fazer a manipulação dos dados. As operações básicas são: insert, select, update e delete.

- Insert: é o comando responsável por inserir os dados nas tabelas do banco de dados.
- Select: é o comando que possibilita selecionar uma quantidade x de dados da tabela.
- Update: é o comando responsável que tem a função de atualizar algum dado da tabela.

- Delete: é o comando responsável por excluir os dados que estão inseridos na tabela.

2.1.3. ACID (Atomicidade, Consistência, Isolamento, Durabilidade)

Uma característica relacionada aos SGBDs é a sua segurança na manipulação dos dados, isso ocorre pelo fato de utilizar-se do paradigma ACID (Atomicity, Consistency, Isolation, Durability). Do portugês Atomicidade, Consistência, Isolamento, Durabilidade.

Segundo ELMASRI (2005, p.403) a atomicidade consiste em uma transação envolvendo duas ou mais partes de dados discretos, ou seja, a transação só será executada totalmente ou não será executada, garantindo que as transações sejam atômicas. A propriedade consistência garante que as transações criem um estado válido dos dados e caso ocorra uma falha os dados retornam ao seu estado antes de ocorrer a transação. A propriedade isolamento é uma transação em andamento ainda não válida que deve permanecer isolada, ou seja, ela deve ficar isolada das demais transações para não sofrer nenhuma interferência. A propriedade durabilidade garante que as mudanças aplicadas no banco de dados mesmo que ocorra uma falha estejam disponíveis e não sejam perdidas.

2.1.4. Normalização

Segundo a Microsoft (2007), normalização é o processo de modelação do banco de dados, está incluído a criação e estabelece a relação entre as tabelas tendo como base as regras estabelecidas para manter a integridade dos dados, tornando um banco de dados mais flexível e eliminando redundâncias.

> O processo de normalização compreende o uso de um conjunto de regras, chamadas de formas normais. Ao analisarmos o banco de dados e verificarmos que ele respeita as regras da primeira forma normal, então podemos dizer que o banco está na “primeira forma normal”. Caso o banco

> respeite as primeiras três regras, então ele está na "terceira forma normal". Mesmo existindo mais conjuntos de regras para outros níveis de normalização, a terceira forma normal é considerada o nível mínimo necessário para grande parte das aplicações. (Microsoft, 2007)

Desta forma, existem algumas regras para fazer a normalização do banco de dados, essas regras são chamadas de " normal form ". As formas normais podem ser vistas como uma análise de determinados esquemas a fim de alcançar as propriedades desejadas, minimizando redundâncias e anomalias (ELMASRI, 2005, p.225).

2.1.5. Limitações

Mesmo com a utilização de um conceito dominante em aplicações de banco de dados, o modelo relacional (MR) apresenta algumas limitações, como por exemplo, não oferece um alto nível de escalabilidade, além disso, impor que as aplicações utilizem de esquemas fixos, consequentemente não será possível estruturar seus dados de acordo com as necessidades individuais.

O modelo relacional é bem complexo levando em consideração todas as suas regras, devido a esse modelo possui chaves que garante os relacionamentos entre as tabelas e seguir regras como a normalização de dados.

Com intuito de suprir essas limitações que tem o modelo relacional, foram desenvolvidas novas tecnologias, entre elas, surgiu o NoSQL, que é uma nova tecnologia para a solução de armazenamento de dados para suprir as necessidades, onde o banco de dados tradicionais são ineficazes.

2.2. Banco de dados NoSQL

Sabemos que existem vários modelos de banco de dados (BD), entretanto com o crescimento elevado de informações, percebeu-se a necessidade de desenvolver uma nova tecnologia para resolver os problemas dos bancos de dados tradicionais (relacional). Um dos fatores para a criação dessa nova tecnologia é a

falta de flexibilidade no armazenamento dos dados e os problemas na estruturação dos bancos de dados.

Segundo Lóscio et al. (2011), o Banco de dados NoSQL foi criado com a função de atender os requisitos do gerenciamento de dados em grande volume, sendo os dados semi-estruturados ou não estruturados.

Com o avanço da web, houve um grande aumento de dados gerando uma necessidade maior de armazenamento e processamento dos dados. Desta forma, a popularização possibilitou o surgimento de novas tecnologias, permitindo uma interação social entre as pessoas, facilitando o compartilhamento de informações de forma online.

Um exemplo para a criação dessa nova tecnologia são as redes sociais, pode ser citado o facebook que é uma rede social onde existe um grande fluxo de dados, onde vários usuários podem publicar textos, vídeos, fotos, utilizar chat, entre outras funcionalidades existentes, desta forma, se tornando um meio de comunicação que apresenta um grande volume de dados, exigindo um bom funcionamento e processamento de todos os dados. Segundo Diana & Gerosa (2010) para auxiliar e suprir essas necessidades surgiu uma nova categoria de banco de dados denominada de NoSQL (Not only SQL).

O termo NoSQL foi utilizado pela primeira vez no final da década de 90, por Carlo Strozzi, com seu Sistema de Gerenciamento de Banco de Dados (SGBD) baseado no Modelo Relacional.

No ano de 2004, segundo Lóscio et al. (2011) a Google lançou o Bigtable, sua própria solução NoSQL que tinha o objetivo de armazenar e gerenciar os dados estruturados em larga escala permitindo a escalabilidade e a alta performance no processamento das consultas, dos processos e dos serviços. Em 2007, a Amazon apresentou o Dynamo, um banco de dados não-relacional criado para ser utilizado por seus serviços. Além da Amazon, o LinkedIn desenvolveu sua própria solução NoSQL denominada de Voldemort, devido ao grande crescimento dos usuários, o desempenho no processamentos das consultas foi afetado, como consequência a empresa desenvolveu a sua própria solução NoSQL, que suporta escalabilidade

horizontal, replicações, particionamento dentre outras funcionalidades. A rede social Facebook desenvolveu um banco de dados e o nomeou de Cassandra que é utilizado para dar suporte a manipulação de um grande volume de dados, posteriormente o Cassandra passou a ser utilizado por outras empresas como Cisco, Digg e Twitter.

2.2.1. Principais características dos bancos de dados NoSQL

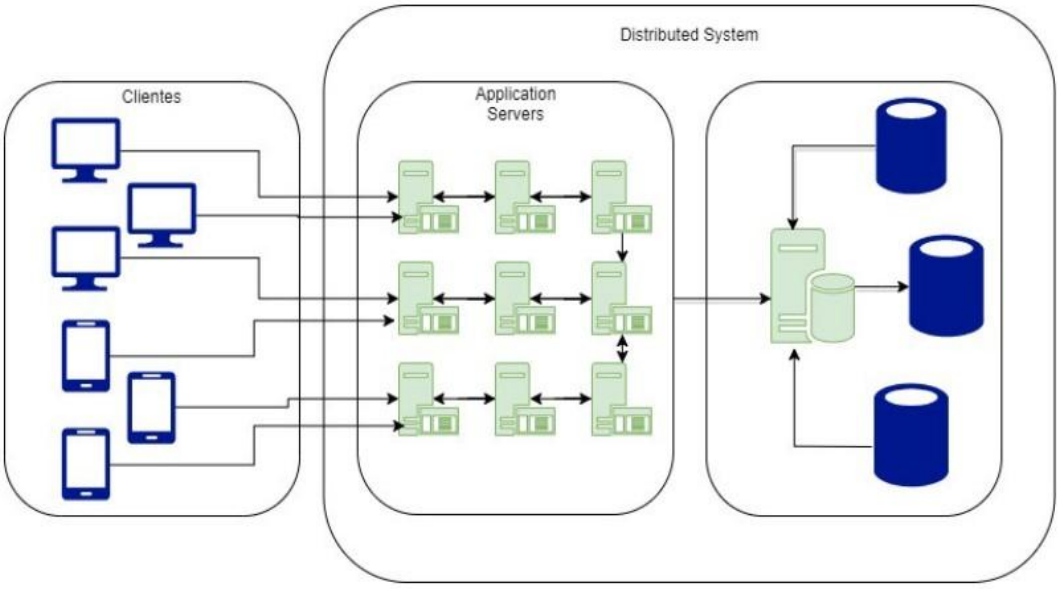
Os bancos de dados NoSQL apresentam características que se diferenciam dos modelos de banco de dados relacionais, fazendo com que sejam adequados para armazenar grande volume de dados, seja ele estruturado ou não-estruturado. A seguir será apresentado algumas dessas características.

2.2.1.1. Escalabilidade horizontal

A partir do momento que o volume de dados aumenta, consequentemente aumenta a necessidade de melhor desempenho e escalabilidade. Segundo Lóscio et al. (2011) uma das soluções para este problema é a escalabilidade horizontal, que tem a função de aumentar o poder de processamento e armazenamento dos dados, esse processo consiste em aumentar o número de máquinas disponíveis para o armazenamento dos dados.

A escalabilidade horizontal é uma solução mais viável porém é necessário diversas threads. Uma opção para se alcançar a escalabilidade horizontal é a utilização de Sharding, que tem a função de dividir os dados em diversas tabelas presentes em diversos nós da rede. Esse processo quebra a lógica de relacionamento utilizado no modelo de banco de dados relacional. Para exemplificar, a figura 1 mostra o processo de aumento do número de máquinas.

Figura 1 - Escalonamento Horizontal



Fonte: https://medium.com/xp-inc

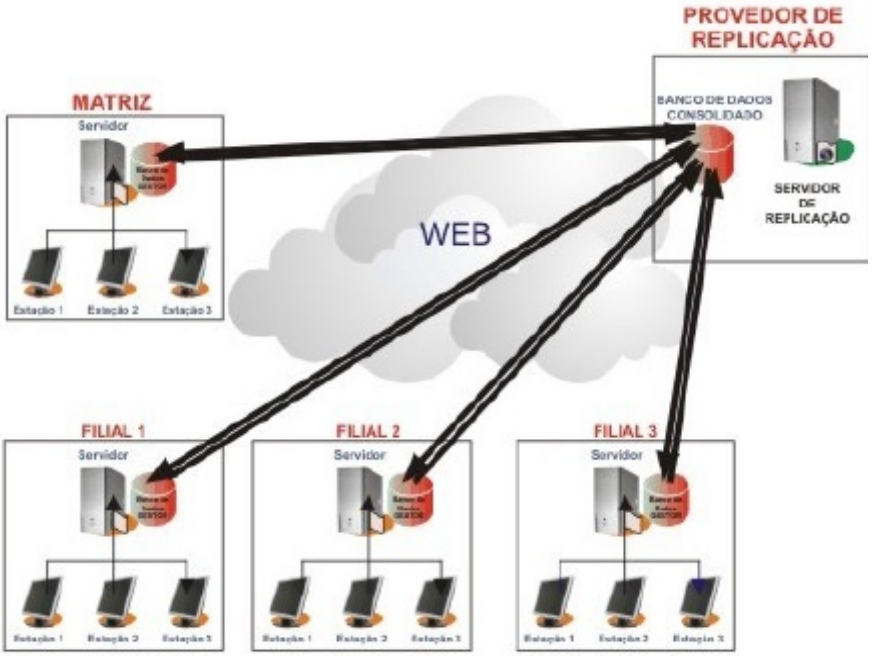
2.2.1.2. Ausência de esquema ou esquema flexível

Segundo Lóscio et al. (2011) uma característica dos bancos de dados NoSQL é a ausência completa ou quase total do esquema que define a estrutura dos dados modelados. Com a ausência de esquema facilita tanto a escalabilidade como também contribui para um aumento da disponibilidade. Entretanto, não há garantia da integridade dos dados, o que em contrapartida ocorre nos bancos relacionais, devido à sua estrutura rígida.

2.2.1.3. Suporte nativo a replicação

Segundo Lóscio et al. (2011) existem outras formas de prover escalabilidade, uma dessas formas é a utilização da replicação. A replicação consiste em diminuir o tempo gasto para recuperar informações. Para prover a replicação um mesmo conjunto de dados são armazenados em vários servidores distintos de forma nativa, diminuindo o tempo e a recuperação dos dados. A figura 2 ilustra a replicação dos dados.

Figura 2 - Replicação de Dados



Fonte:https://svitrini.com.br/servidor-de-banco-de-dados-replicados

Segundo Brito (2010) essa técnica possibilita que o sistema esteja sempre disponível e, caso algum servidor falhe, o acesso aos dados são desviados para outro servidor.

2.2.1.4. API simples para acesso aos dados

Segundo Lóscio et al. (2011) o objetivo da solução NoSQL é prover uma forma eficiente de acesso aos dados, oferecendo alta disponibilidade e escalabilidade, ou seja, o foco não está em como os dados são armazenados e sim como poderemos recuperá-los de forma eficiente. Para isto, é necessário que APIs sejam desenvolvidas para facilitar o acesso a estas informações, permitindo que qualquer aplicação possa utilizar os dados do banco de forma rápida e eficiente.

2.2.2. Tipos de Modelo de Dados no NoSQL

Esta seção descreve os modelos de dados usados no banco de dados NoSQL, os tipos de modelo de dados são: banco de dados baseado em

chave-valor, orientados a documentos, banco de dados orientado a grafos e colunas.

Segundo Sadalage e Fowler (2013, p.41), o termo "Modelo de Dados" serve para se referir qual o gerenciador do banco de dados que irá organizar seus dados. A tecnologia NoSQL possui diferentes modelos, os quais foram divididos em quatro categorias: Documento, Chave-valor, Colunas e Grafos.

2.2.2.1. Sistemas orientados a Documentos

Segundo Lóscio et al. (2011) o modelo orientado a documentos possui um conjunto de documentos e em cada um deles um conjunto de campos (chaves) e os valores deste campo. Como principais aplicações que adotam o modelo orientado a documentos destacamos CouchDB, MongoDB, RavenDB, Scalaris, Jackrabbit (KUBACKI, 2010 apud Delgado 2011,p. 39).

Sabe-se que existem alguns cenários para os quais os armazenamentos de documentos são uma solução apropriada (Sadalage e Fowler 2013, p.145), a citar:

- **Sistema de Gerenciamento de Conteúdo (CMS), plataformas de blog:** Os bancos de dados em documentos não tem esquemas pré definidos e geralmente são usados documentos em formato JSON, eles são bem usados em sistemas de gerenciamento de conteúdo ou aplicativos voltado para publicação em websites, ajudando no gerenciamento dos conteúdos tais como comentários, registros perfis e documentos que possam ser visualizados pelos usuários na web.

- **Análise web ou em tempo real (analytics):** Os dados podem ser armazenados para análise em tempo real, certa vez que parte desse documento possa ser atualizado. Desta forma, novas métricas podem ser adicionadas com facilidade, sem ter alteração no esquema.

Na Figura 3 temos um exemplo de documento representando um post de um blog, onde os campos foram definidos por: Assunto, Autor, Data, Tags e Mensagem.(Lóscio, 2011, p.8).

Figura 3 - Exemplo de documento

ID: P001
Assunto: "Eu gosto de laranjas"
Autor: "Hélio"
Data: "27/01/2011"
Tags: ["laranjas", "suco", "plantas"]
Mensagem: "Hoje estou com vontade de tomar suco de laranja!"

Fonte: LÓSCIO, 2011, p.9

2.2.2.2. Sistema baseado em Chave-valor

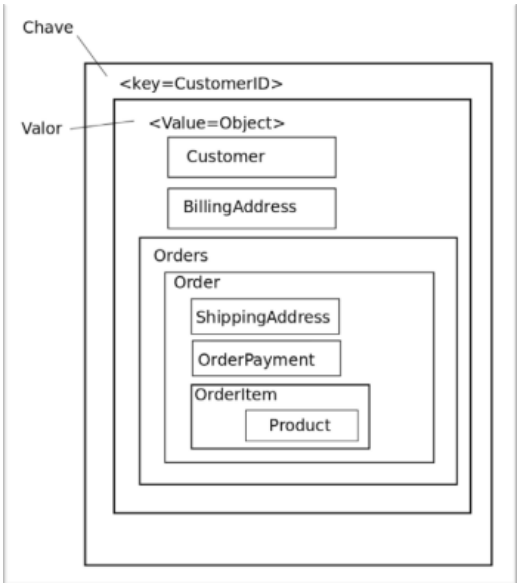
Segundo Lóscio et al. (2011) este modelo é considerado simples pelo fato de permitir a visualização do banco de dados como uma grande tabela hash. Desta forma, o banco de dados é composto por um conjunto de chaves que estão associados a um único valor.

Delgado (2013, p.40) afirma que, embora os sistemas chave-valor sejam os mais simples, são os que suportam maior volume de dados e potenciais de escalabilidade. Segundo Toth (2011 apud Almeida e Brito 2012), o modelo chave-valor na maioria dos casos apresenta uma interface composta principalmente pelos seguintes métodos:

- Put(chave, valor) – representa o método que insere um registro e garante maior velocidade a operação;
- Get(chave) – responsável por recuperar um registro, não oferece opções para buscas mais complexas;

Existem vários tipos de banco de dados do tipo chave- valor, mas os mais populares segundo Sadalage e Fowler (2013, p.124) são o Riak, Redis, Memcached DB e suas variáveis, Amazon DynamoDB, entre outros. A Figura 3 demonstra como é a modelagem de um sistema baseado em chave valor.

Figura 4 - Dados armazenados em chave-valor



Fonte:SADALAGE E FOWLER, 201, p.67

Recursos dos depósitos chave-valor:

- **Consistência:** é aplicável apenas às operações em uma única chave; gravações otimistas podem ser executadas, mas são muito caras para implementar.

- **Transações:** Diferentes produtos no armazenamento de chave-valor têm diferentes especificações de transação; Não há garantia nas transações.

- **Recursos de consultas:** Todos os armazenamentos de chave-valor podem ser consultados pela chave; Realizar consultas por chave otimiza muito o tempo de consulta.
- **Estrutura de dados:** banco de dados chave-valor não se importa com o que é armazenado na parte do valor do par chave-valor; o valor pode ser um blogue, texto, JSON, XML e etc.
- **Escalabilidade**: muitos armazenamentos de chave-valor podem ser escalados utilizando a fragmentação.

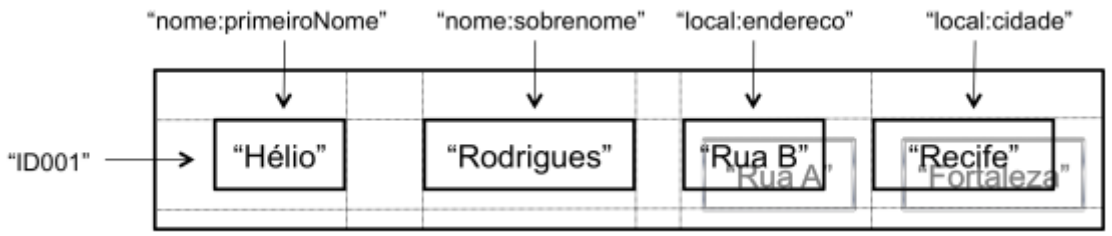
2.2.2.3. Sistemas orientados a Colunas

Neste modelo existe uma grau de complexidade maior comparado com o modelo chave-valor, neste caso, muda-se o paradigma de orientação a registro ou tuplas (como no modelo relacional) para orientação a atributos ou colunas (modelo NoSQL) (LÓSCIO et al., 2011). No modelo baseado em família de colunas os dados são indexados por uma tripla (linha, coluna e timestamp), onde linhas e colunas são identificadas por chaves e o timestamp permite diferenciar múltiplas versões de um mesmo dado (LÓSCIO et al., 2011, p. 7).

Como principais soluções que adotam esse modelo destaca-se o BigTable da Google, pois permite o particionamento dos dados, além de oferecer forte consistência e outro exemplo é o Cassandra, que foi desenvolvido pelo Facebook e é baseado no Dynamo.

Observe o exemplo da Figura 5. que modela o conceito de amigos, onde primeiroNome e sobrenome são colunas pertencentes à família de colunas denominada nome. De maneira semelhante, as colunas endereço, cidade e estado pertencem à família local. Observe que para a linha 001, o amigo Hélio tem diferentes endereços, onde para cada um deles tem um timestamp correspondente (LÓSCIO et al., 2011, p. 7).

Figura 5 - Exemplo de modelo de dados baseado em colunas



Fonte: LÓSCIO, 2011, p.8

Este modelo de dados surgiu com o BigTable da Google, por isso é comum falar sobre o modelo de dados BigTable. Dentre as características deste modelo temos a permissão do particionamento dos dados, além de oferecer forte consistência, mas não garante alta disponibilidade. Outras soluções surgiram após o

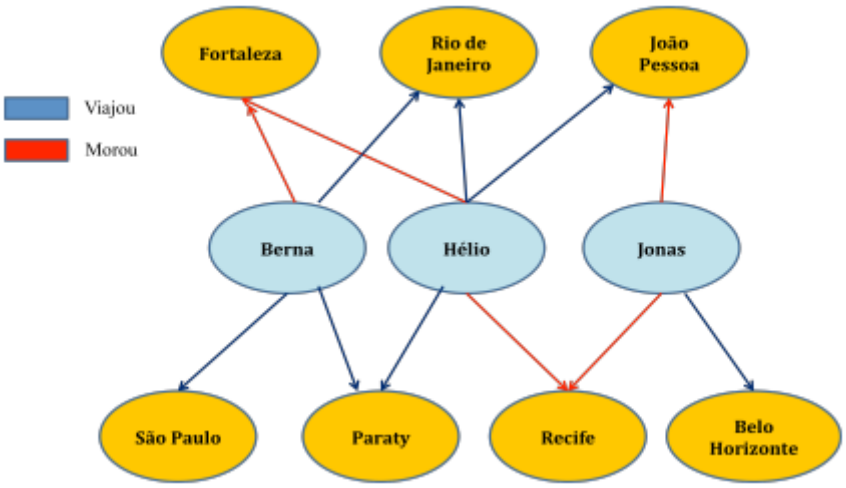
Bigtable, dentre elas o Cassandra, desenvolvido pelo Facebook e baseado no Dynamo (LÓSCIO et al., 2011, p. 8).

2.2.2.4. Sistemas orientados a Grafos

O modelo orientado a grafos, diferente dos outros modelos, possui três componentes básicos, como: os nós (são as vértices do grafo), os relacionamentos (são chamados de arestas) e as propriedades (também chamadas de atributos) dos nós e relacionamentos. A vantagem da utilização deste modelo fica visível a partir do momento que são necessárias consultas mais complexas exigidas pelo usuário. Alguns exemplos de banco de dados que são baseados em grafos: Neo4j, AllegroGraph, InfiniteGraph e OrientDB.

A figura 6 apresenta um exemplo de grafo que representa os dados de uma aplicação que deseja manter informações de locais de viagens e locais onde as pessoas residem.

Figura 6 - Estrutura baseada em grafos



Fonte: LÓSCIO, 2011, p.9

Aplicativos de alta disponibilidade, onde a minimização da inatividade é fundamental, podem utilizar um banco de dados orientado a colunas, como o Cassandra. Aplicações que exigem alto desempenho em consultas com muitas junções, podem adotar um banco orientado a grafos. A importância de conhecer cada solução é adotar aquela que for mais adequada poderá contribuir para a diminuição do custo de criação do banco de dados, bem como para o aumento da eficiência no processamento dos dados (LÓSCIO, 2011, p.9).

2.2.2.4 Sistemas de Gerenciamento de Banco de Dados NoSQL

A tabela a seguir descreve alguns exemplos de SGBDs que foram desenvolvidos por grandes empresas para auxiliar no armazenamento de um grande fluxo de informações.

Tabela 1 - Características SGBDs e casos de sucesso

SGBDs	Características	Casos de sucesso
BigTable	Foi desenvolvido para distribuir dados por centenas de servidores e escalar com conjuntos de dados de até 1 petabyte, o BigTable tem código aberto.	O BigTable foi desenvolvido pela Google, sua própria solução NoSQL. Esta solução permite a escalabilidade de recursos, bem como alta performance no processamento das consultas, dos processos e dos serviços.
Dynamo	Foi desenvolvido em 2007, com intuito de oferecer armazenamento de dados em grande escala utilizando-se de chave-valor.	A Amazon desenvolveu uma solução NoSQL, o Dynamo. Como resultado da adoção desta nova tecnologia, diversos serviços da Amazon têm se mantido disponíveis em 99,99% das requisições realizadas.
Cassandra	É um sistema de banco de dados distribuído, escalável, que foi	O Facebook desenvolveu o Cassandra, uma solução NoSQL.

	desenvolvido na plataforma Java, reunindo a arquitetura do Dynamo, da Amazon e o modelo de dados do BigTable, do Google.	Inicialmente criado para otimização do sistema de busca do Facebook, atualmente o Cassandra é utilizado para dar suporte à replicação, detecção de falhas, armazenamento em cache dentre outras funcionalidades.
MongoDB	Foi desenvolvido com o código fonte aberto em linguagem de programação C ++, alta performance e utilizando-se de armazenamento orientado a documentos.	O seu desenvolvimento começou em 2007 pela 10gen, atual MongoDB Inc. Oferecendo alta disponibilidade. Desta forma, o jogo Cartola FC da Globo, com o crescimento do game a equipe de desenvolvimento decidiu por migrar o banco de dados do Mysql para MongoDB.

Fonte: (FARIAS, 2014, p.30)

2.3. Banco de Dados Relacional x Banco de Dados NoSQL

Segundo Lóscio (2011) a tecnologia relacional foi proposta na década de 70, quando as aplicações de banco de dados caracterizavam-se por lidar com dados estruturados, ou seja, dados que possuem uma estrutura fixa e bem definida.

A diferença do NoSQL é que utiliza escalonamento horizontal e tem maior facilidade de distribuição dos dados, ou seja, com um aumento de dados, aumenta-se o número de servidores, que podem ser ou não de alta performance, barateando e otimizando o armazenamento. É justamente neste ponto, escalabilidade, que o NoSQL destaca-se, pois este novo paradigma, segundo Lóscio et al., (2011, p. 3 apud Almeida e Brito 2012), atende “requisitos de alta

escalabilidade necessários para gerenciar grandes quantidades de dados, bem como para garantir a alta disponibilidade dos mesmos". Para Delgado (2011, p. 28) a tecnologia NoSQL propõe um novo modelo de organização de dados, que permite que aplicações possuam uma estrutura mais flexível, possibilitando que sejam altamente escaláveis, além de possibilitar que os dados sejam manipulados livres de regras, podendo cada SGBD seguir um modelo de organização.

A Tabela 2 permite uma análise comparativa entre as classes de banco de dados, a partir do teorema CAP (Consistency, Availability e Partition Tolerance), que em português, significa: consistência, disponibilidade e tolerância ao particionamento (na Tabela 2 representada pelo escalonamento).

NoSQL não oferece a propriedade de consistência como em bases relacionais (que garantem a integridade dos dados), apresenta como pontos fortes o escalonamento e a disponibilidade (conforme visto na Tabela 2).

Tabela 2 - Análise comparativa Modelo Relacional e NoSQL

	Modelo Relacional	**NoSQL**
Escalabilidade	Possível, mas complexo. Devido à natureza estruturada do modelo, a adição de forma dinâmica e transparente de novos nós no grid não é realizada de modo natural.	Uma das principais vantagens desse modelo. Por não possuir nenhum tipo de esquema pré-definido, o modelo possui maior flexibilidade o que favorece a inclusão transparente de outros elementos.
Consistência	Ponto mais forte do modelo relacional. As regras de consistência presentes propiciam um maior grau de rigor quanto à consistência das informações.	Realizada de modo eventual no modelo: só garante que, se nenhuma atualização for realizada sobre o item de dados, todos os acessos a esse item devolverão o último valor atualizado.

Disponibilidade	Dada a dificuldade de se conseguir trabalhar de forma eficiente com a distribuição dos dados, esse modelo pode não suportar a demanda muito grande de informações do banco.	Outro fator fundamental do sucesso desse modelo. O alto grau de distribuição dos dados propicia que um maior número de solicitações aos dados seja atendido por parte do sistema e que o sistema fique menos tempo disponível.

Fonte: (BRITO, 2010 , p.5)

2.4. Trabalhos relacionados

Existem vários aspectos para serem mensurados sobre o desempenho do Banco de Dados (BD). Atualmente os cenários da web 2.0 tem se mostrado problemáticos devido ao gerenciamento acentuado de dados, pois não se mostram mais tão eficientes como antes.

Para verificar a problemática que os Sistemas de Gerenciadores de Banco de Dados (SGBDs) relacionais enfrentam para gerenciar sistemas que dispõem de um grande volume de dados, tem-se o trabalho de Brito (2013) que mostra uma análise comparativa fazendo uso de uma nova geração de Banco de Dados conhecida como NoSQL versus SGBDs Relacionais. Esta análise foi motivada principalmente pela questão da escalabilidade do sistema como uma alternativa para aplicar em sistemas que têm o crescimento cada vez mais intensificado de volume de dados.

Outro projeto que faz alusão ao trabalho citado anteriormente apresentando um diferencial como solução para manipulação de grande quantidade de volume de dados é o projeto de Soares (2012). Para Soares (2012, p. 17) com o crescimento e diversidade de informações geradas a todo tempo, os modelos relacionais nem sempre são a melhor alternativa para manipulação de dados. Isso ocorre por diversos fatores, como a forma de armazenamento dos dados em disco, a forma com que o executor de consultas recupera as informações e monta as tabelas e o método de compressão utilizado.

Esse trabalho faz uma análise comparativa de desempenho com as principais operações de banco de dados, usando um cenário que usa de muita junção no modelo relacional, realizando testes em variados volumes de dados que faz alusão ao trabalho de Farias (2014). Brito (2013) afirma que devido ao intenso crescimento do número de aplicações, recursos e tudo o que se refere a sistemas computacionais nas últimas décadas, o volume de dados associados a tais tipos de sistemas também teve um ritmo de crescimento acelerado.

2.5. Considerações finais

Neste capítulo foram apresentadas as principais propriedades dos modelos relacionais, suas características e limitações. Além de uma introdução a NoSQL, o que é, os principais modelos de dados adotados, as principais diferenças em relação ao modelo relacional e suas principais características.

No próximo capítulo serão apresentados vários testes de desempenho em três banco de dados distintos, que utilizam modelo de dados diferentes. Os testes são quantitativos e medem o tempo de resposta de operações como INSERT, DELETE, UPDATE e SELECT.

3. Desenvolvimento

O desempenho de um sistema na engenharia de software é um atributo que requer muito cuidado, pois fatores como hardware, linguagem de programação, algoritmo, banco de dados, podem influenciar na performance do sistema. Por exemplo, um sistema que possui um banco de dados relacional, com várias tabelas relacionadas, provavelmente exigirá muito dos recursos de junções.

Foi criado um cenário controlado de dados que serviu como base para todo o desenvolvimento. A partir desse cenário foram feitas as modelagens dos dados para cada banco de dados, MySQL e MongoDB, e realizados testes de inserção, alteração, pesquisa e exclusão de dados. O objetivo dos testes foi comparar o desempenho de cada operação em cada BD. Foram escolhidos esses modelos de dados, pois acredita-se que estes são os mais viáveis para modelar o caso de uso proposto.

3.1 Ambiente de Testes

O ambiente de testes usado para comparar os tempos de execução de instruções SQL com os comandos dos bancos NoSQL contou com um computador equipado com as configurações descritas na tabela 3 .

Tabela 3 - Configurações do computador

Dispositivo	Descrição
Processador	Intel(R) Core(TM) i5-7400 CPU@3.00 GHz
Memória RAM	16 GB
HD	1 TB SATA
Sistema operacional	Windows 10 Pro 64 bits

3.2 Cenário

O cenário fictício criado, serviu como base para a criação do banco de dados das duas soluções. A seguir pode-se observar a descrição deste cenário.

“A barbearia Barber Club é uma barbearia especializada em todos os tipos de cabelos e oferece várias opções de cortes e tratamentos únicos para os seus clientes. Todos os atendimentos são feitos através de agendamentos, para que seja concluído o seu agendamento o cliente deve fazer o seu cadastro, caso já seja um cliente de longa data o mesmo deve somente entrar em contato para verificar qual colaborador e horários estão disponíveis.

A Barber Club precisa de um sistema de gerenciamento onde ela possa ter acesso aos dados de seus clientes e colaboradores para poder gerenciar os atendimentos com dia e hora marcada e não haver nenhum tipo de inconveniente e poder prestar o seu bom atendimento.

3.3 Arquitetura do sistema

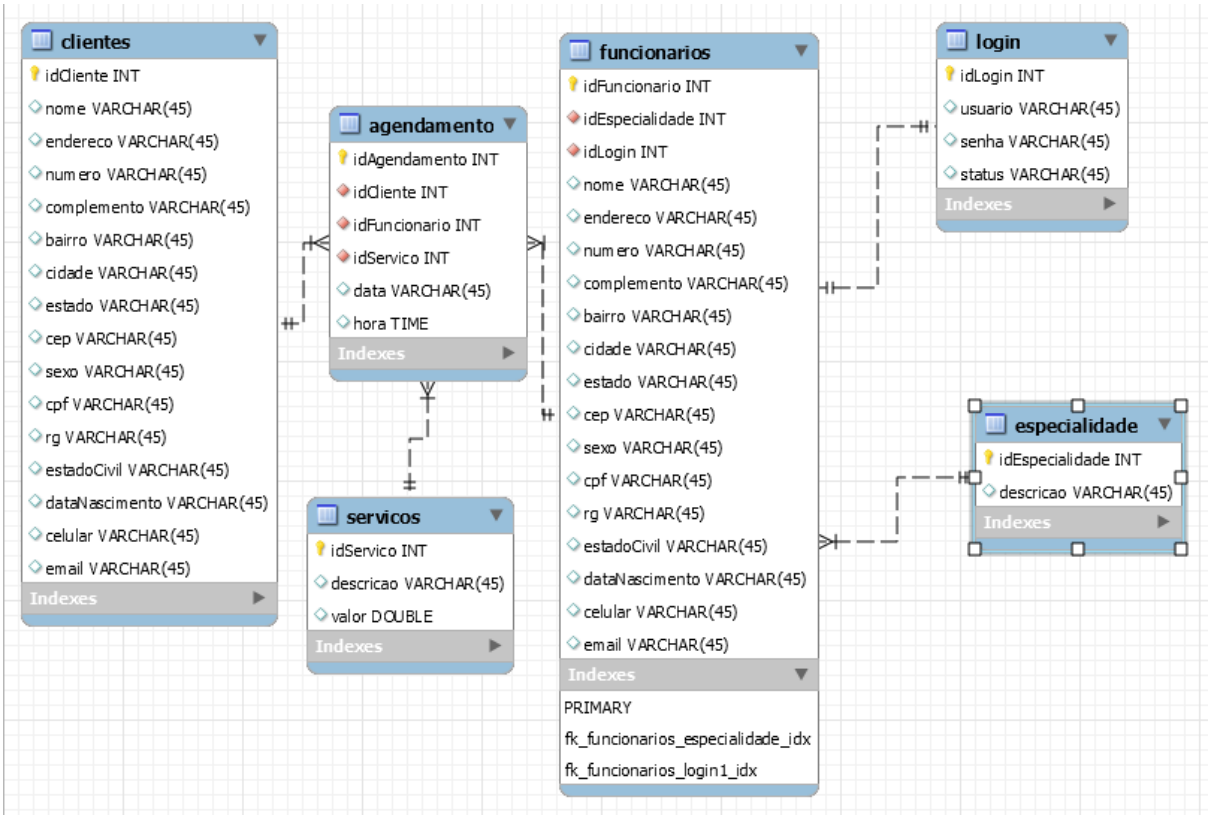
Os dados que alimentaram os bancos de dados foram dados fictícios. Desta forma para auxiliar nas operações e poder adquirir o tempo de execuções da mesma, criou-se uma aplicação a fim de realizar os testes. Para a criação desta aplicação foi utilizado a linguagem de programação Java e a IDE NetBeans, pelo fato de possuir API´s que atende os dois modelos de banco de dados utilizados. Portanto, o tempo decorrido para cada operação incluirá a camada de aplicação java, pois os testes não foram feitos diretamente nos bancos de dados.

Foi criada uma interface DAO com as operações a serem realizadas, pelo menos três classes implementam essa interface, uma para a conexão e lógica aplicada junto com o MySQL, outra para o MongoDB. Para fazer a comunicação entre o banco e a aplicação, foram usadas as API 's java fornecidas por cada BD. Dessa forma, o cenário fica o mais próximo possível do tempo gasto por cada operação do Banco de Dados.

3.4 Modelo relacional

Na Figura 7 é apresentado o diagrama de modelo de entidade e relacionamento (MER) que descreve os requisitos de dados da aplicação, onde cada cliente possui uma chave primária, um nome e dentre outros atributos; cada agendamento, funcionário, serviços, especialidades e login possui uma chave primária e seus determinados atributos, cada atributo contendo seu determinado tipo (Tipo determina qual dado será guardado, por exemplo “nome” ele é do tipo VARCHAR, isso significa que na tabela cliente o campo nome sempre irá suportar letras ou números).

Figura 7 - Modelo entidade e relacionamento



3.4.1 Modelo NoSQL

O paradigma NoSQL é livre de esquema, ele não exige uma estrutura prévia para utilização da aplicação e a estrutura pode ser alterada livremente em tempo de execução, como foi visto anteriormente. Segundo Lennon (2013), o MongoDB armazena dados dentro de documentos semelhantes a JSON (usando BSON, uma versão binária de JSON), que retém os dados usando pares de chave/valor.

Figura 8 - Modelo orientado a documento

```
1  {
2      "_id" : ObjectId("621edb3f970dba1c4d078869"),
3      "cliente" : "cauan",
4      "funcionario" : "caique",
5      "servico" : "corte",
6      "data" : "10-03-2022",
7      "hora" : "18:00"
8  }
```

O documento Json demonstra um agendamento de um cliente de nome Cauan, que solicitou um serviço com data e hora marca que será efetuado pelo funcionário Caique.

3.5 Realização dos testes

Nessa seção é descrito como foram feitos os testes, onde primeiramente se povoou o banco de dados através da operação de INSERT, utilizando as modelagens descritas na seção anterior, e também os testes de SELECT, UPDATE e DELETE, os testes medem a diferença entre os tempos de execução de todas essas operações nos banco de dados MySQL e MongoDB.

Para medir o tempo de execução de cada comando, foi utilizado o método currentTimeMillis() da classe java.lang.System, o qual retorna o horário atual do sistema em milissegundos. Sendo assim, a aplicação armazena em uma variável o tempo inicial, em seguida executa a operação, captura o tempo final e não realiza a diferença entre o tempo final e o inicial, obtendo assim o tempo de execução, todos os testes foi feito 10 vezes e feito uma média para chegar no tempo de execução.

3.5.1 INSERT

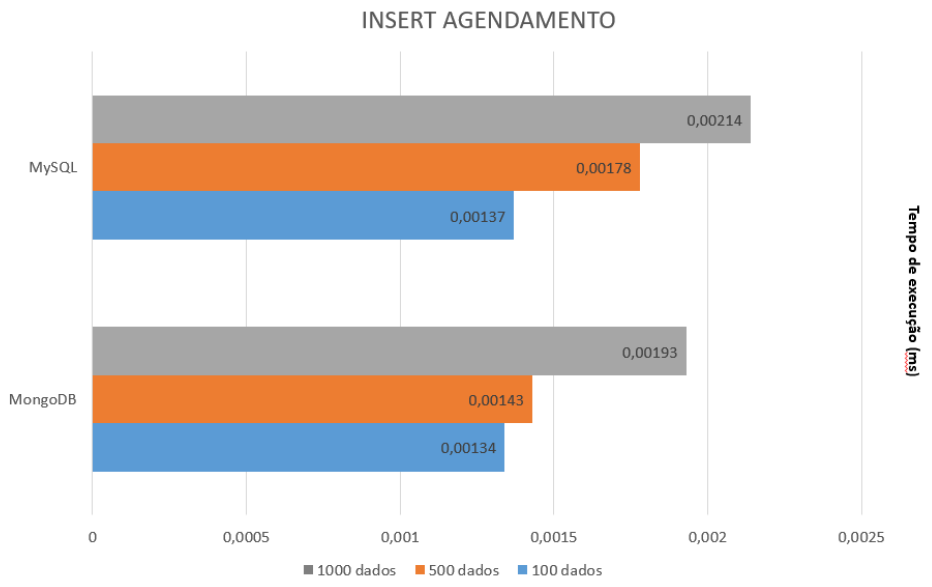
Para implementação do método utilizando o MySQL, a inserção utilizou o recurso de junções entre as tabelas clientes, servicos e funcionarios, em seguida fez uma série de consultas simples às tabelas a fim de inserir os dados do agendamento. O quadro apresenta a alteração realizada.

Quadro 1 - Insert MySQL realizado no NetBeans

```
String sqlInserir = "INSERT INTO agendamento("
        + "idAgendamento, idCliente, "
        + "idFuncionario, idServico, data, hora) "
        + " VALUES (?,?,?,?,?,?)";
```

O gráfico da Figura 8 mostra a comparação do tempo de execução entre as instruções de inserção para o MySQL e MongoDB.

Figura 9 - Gráfico Insert



O que se destaca no gráfico da Figura 9 é que o MongoDB possui um desempenho superior, ou seja, gasta menos tempo para inserir os dados do que o MySQL. Uma possível explicação para este resultado é que definiu-se o esquema

do MongoDB em apenas um documento agregado, isso significa que todos os dados de um cadastro de agendamento estão modelados em um único documento.

3.5.2 DELETE

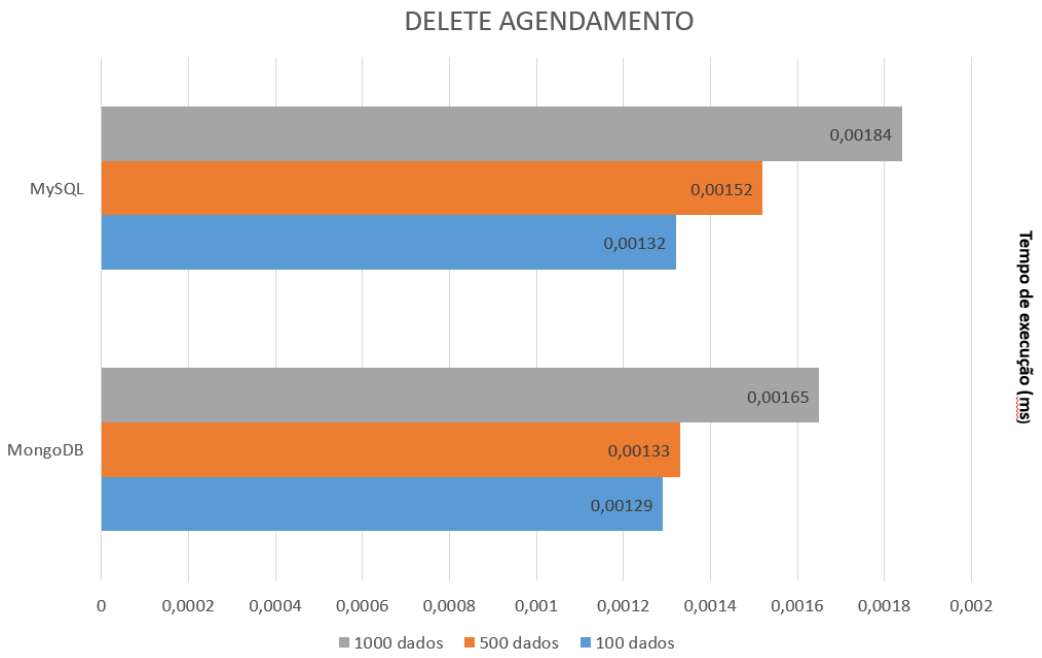
Para implementação do método utilizando o MySQL, o delete utilizou o recurso de junções entre as tabelas clientes, servicos e funcionarios, em seguida fez uma série de consultas simples às tabelas a fim de excluir os dados do agendamento. O quadro apresenta a alteração realizada.

Quadro 2 - Delete MySQL realizada no NetBeans

```
String sqlExcluir = "DELETE FROM agendamento WHERE idAgendamento=?";
```

O gráfico da Figura 10 mostra a comparação do tempo de execução entre as instruções de exclusão para o MySQL e MongoDB.

Figura 10 - Gráfico Delete



O MongoDB se mostrou mais eficiente que o MySQL, mostrando que o desempenho não sofre muito com o aumento da quantidade de dados na base, a operação DELETE mesmo tendo 100 ou 500 dados na sua base de dados o tempo de execução foi bem próximo chegando a quase ser o mesmo.

3.5.3 UPDATE

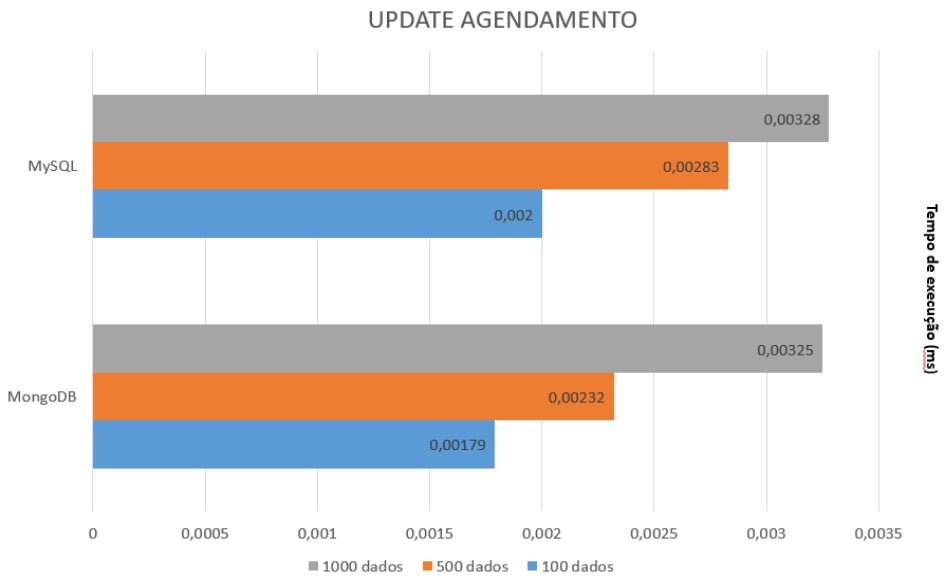
Para implementação do método utilizando o MySQL, o update utilizou o recurso de junções entre as tabelas clientes, servicos e funcionarios, em seguida fez uma série de consultas simples às tabelas a fim de alterar os dados do agendamento. O quadro apresenta a alteração realizada.

Quadro 3 - Update MySQL realizada no NetBeans

```
String sqlAlterar ="UPDATE `agendamento` SET `idAgendamento`=?,`idCliente`=?,"
        + "`idFuncionario`=?,`idServico`=?,`data`=?,`hora`=? "
        + "WHERE `agendamento`.`idAgendamento` = 1";
```

O gráfico da Figura 11 mostra a comparação do tempo de execução entre as instruções de atualização para o MySQL e MongoDB.

Figura 11 - Gráfico Update



O MongoDB apresentou um pequeno ganho de desempenho porém foi bem parelho em relação ao tempo de execução do MySQL, podemos concluir que mesmo com uma quantidade considerável de dados o MySQL conseguiu ter um bom desempenho.

3.5.4 SELECT

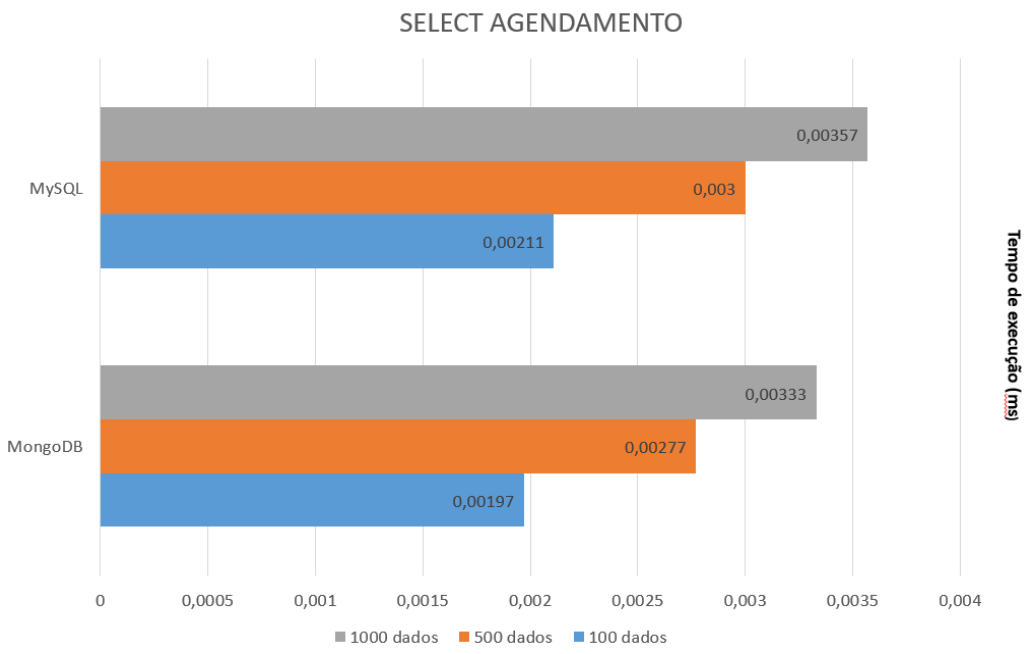
Para implementação do método utilizando o MySQL, a consulta utilizou o recurso de junções entre as tabelas clientes, servicos e funcionarios, em seguida fez uma série de consultas simples às tabelas a fim de recuperar os dados do agendamento. O quadro apresenta a consulta realizada.

Quadro 4 - Select MySQL realizada no NetBeans

```
String sqlListar = "SELECT * FROM agendamento";
```

O gráfico da Figura 12 mostra a comparação do tempo de execução entre as instruções de seleção para o MySQL e MongoDB.

Figura 12 - Gráfico Select



O MongoDB apresentou um ganho de desempenho em relação ao tempo de execução do MySQL, porém a diferença entre os dois modelos é pequena, levando em consideração o cenário de teste essa diferença não é perceptível .

3.6 Softwares utilizados

As versões dos bancos de dados e os softwares utilizados na realização dos testes anteriores são descritas na tabela a seguir:

Tabela 4 - Softwares utilizados

Software	Versão
Mysql	8.0 CE
MongoDB	5.0.6
NetBeans	8.2
Java	8.0.23

3.7 Análise dos dados

Analisando os dados apresentados é possível tirar algumas conclusões em relação à inserção, exclusão, atualização e consulta dos dados de acordo com todo o ambiente de testes levando em consideração o seu escopo. Entretanto, é importante ressaltar que os dados apresentados são apenas uma base do desempenho geral e deve ser analisada a aplicação levando em consideração o contexto em que foi aplicada.

Com base nos gráficos é possível concluir que, na média geral , o MongoDB obteve um desempenho melhor que o MySQL. Os principais motivos para justificar tal desempenho foi o uso de um único documento para representação das tabelas do modelo relacional onde necessitava de dados de outras tabelas, porém mesmo com esses resultados não significa que o modelo NoSQL seja superior ao relacional.

O que pode-se observar foi a limitação que o MySQL impôs diante do modelo criado inicialmente, isso enfatiza que o NoSQL, apesar da flexibilidade para alterar o modelo dos dados, não é totalmente livre de esquemas, o modelo inicial do projeto deve ser bem elaborado para satisfazer as necessidades da aplicação, tudo isso deve ser levado em consideração ao montar o esquema dos dados.

4. Considerações finais

Os diversos produtos existentes no mercado para o armazenamento de dados, conhecidos como bancos de dados, trabalham com diversas tecnologias. Dentre os mais populares estão os produtos que implementam o modelo relacional, como o SQLServer, Oracle, MySQL e o PostgreSQL. Recentemente novas abordagens para o armazenamento e manipulação de dados foram apresentadas e agrupadas pela sigla NoSQL, incluindo produtos como o MongoDB, Cassandra, Redis, CouchDB, Neo4J, entre outros.

O presente trabalho apresentou uma comparação de desempenho tendo como base o tempo de execução das instruções INSERT, DELETE, UPDATE e SELECT, operações das quais foram executadas nos bancos de dados SQL e NoSQL. Os bancos de dados comparados foram o MySQL e o MongoDB.

Todos os dados utilizados para fazer as comparações foram dados fictícios, é muito importante ressalta que vários fatores pode influenciar no tempo de execução, como por exemplo o processador, memória ram, versão do software utilizada, em caso de um sistema onde existem vários usuários conectados ao mesmo tempo fazendo requisições a sua base de dados. Todos os exemplos citados entre vários outros devem ser levados em consideração, em certos computadores ou aplicações, o tempo de execução mostrado no decorrer deste trabalho irá ter variações.

Contudo, é preciso sempre levar em consideração diversos fatores quando se decide adotar ou não a utilização de um software. Portanto, os resultados obtidos não são suficientes para influenciar a escolha das tecnologias a adotar. Mas, os profissionais que trabalham com bancos de dados e programação possuem um ponto de partida para justificar futuras pesquisas e testes que avaliem o desempenho dos bancos de dados NoSQL e SQL.

REFERÊNCIA BIBLIOGRÁFICAS

BRITO, R. W.. (2013). **Bancos de Dados NoSQL x SGBDs Relacionais:Análise Comparativa***. In:. Infobrasil TI & TELECOM , Fortaleza – CE. Disponível em: <http://docplayer.com.br/433629-Bancos-de-dados-nosql-x-sgbds-relacionais-analise-comparativa.html>. Acesso em: 13 de Fevereiro de 2022, às 19h30min.

COSTA, E. R. de.. **Banco de Dados Relacional**. São Paulo, 2011. 64p. Trabalho de conclusão de curso (Tecnólogo em Processamento de Dados) - Faculdade de Tecnologia de São Paulo, São Paulo, SP. Disponível em: <http://www.fatecsp.br/dti/tcc/tcc0025.pdf>. Acesso em: 14 de Janeiro de 2022, às 23h.

DIANA, M. D.; GEROSA, M. A. **Nosql na web 2.0: Um estudo comparativo de bancos não relacionais para armazenamento de dados na web 2.0**. In: IX Workshop de Teses e Dissertações em Banco de Dados. [S.l.: s.n.], 2010. v. 9.

DELGADO, A. G.. **Apresentação da Tecnologia NoSQL através de um estudo de caso do Banco de Dados Apache Cassandra**. João Pessoa, 2011. 82p. Trabalho de conclusão de curso (Graduação em Bacharelado em Ciência da Computação) – Centro Universitário de João Pessoa – UNIPE, João Pessoa – PB. Disponível em: Acesso em: 13 de Fevereiro de 2022, às 21h02min.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 4ª Ed. São Paulo: Pearson Addison Wesley, 2005. cap. 1, p. 16.

FARIAS, F. A. M. **Avaliação de Desempenho entre Bancos de Dados Relacionais e NoSQL.** Rio Tinto - PB, 2014. 62p. Trabalho de conclusão de curso - Universidade Federal da Paraíba, Rio Tinto, PB. Disponível em: <https://si.dcx.ufpb.br/wp-content/uploads/2015/12/Fred-Augusto-de-Melo-Farias.pdf>. Acesso em: 7 de Fevereiro de 2022, às 23h07min.

LEITE, G.S. **Análise Comparativa do Teorema CAP Entre Bancos de Dados NoSQL e Bancos de Dados Relacionais**. Fortaleza, 2010. 49p. Trabalho de conclusão de curso (Graduação em Ciência da Computação) – Faculdade Farias Brito,Fortaleza–CE. Disponível em:<http://fbuni.edu.br/sites/default/files/tcc-20102-gleidson-sobreira-leite.pdf>. Acesso em: 23 de Fevereiro de 2022, às 15h16min.

LÓSCIO, B. F.; OLIVEIRA, G. R.; PONTES J. C. de S..(2011). **NoSQL no Desenvolvimento de aplicações Web Colaborativas**. In:. VIII SBSC, Paraty, RJ. Disponível: <https://www.researchgate.net/profile/Bernadette-Loscio/publication/268201466_NoSQL_no_desenvolvimento_de_aplicacoes_Web_colaborativas/links/576aa72008aef2a864d1ef8c/NoSQL-no-desenvolvimento-de-aplicacoes-Web-colaborativas.pdf>. Acesso em: 09 de Fevereiro de 2022, às 21h 19 min.

LENNON, Joe; **A arte da programação**, 2013. Disponível em: <http://alfavillecode.blogspot.com/>. Acesso em: 26 de Janeiro de 2022, às 01h06min.

MACÁRIO, C. G. do N.; BALDO, S. M.. **O Modelo Relacional**. Trabalho como parte da avaliação do curso MO-410 (curso: Introdução a Banco de Dados) - Instituto de Computação da Unicamp, Campinas, SP, 2005. Disponível em: <http://www.ic.unicamp.br/~geovane/mo410-091/Ch03-RM-Resumo.pdf>. Acesso em: 17 de Janeiro de 2022, às 22h10min.

NOSQL. **NOSQL Databases**. 2010. Disponível em: <http://nosql-database.org/>. Acesso em: 15 de Janeiro de 2022, às 23h.

SADALAGE, J.; FOWLER, M. **NoSQL: Um Guia Conciso para o Mundo Emergente da Persistência Poliglota**, [tradução Acauan Fernandes]. Novatec Editora Ltda, 2013. 216p. ISBN 978-85-7522-338-3.

SOARES, B. E.. U**ma avaliação experimental de desempenho entre Sistemas Gerenciadores de Banco de Dados Colunares e Relacionais**. Cascavel, 2012. 102p. Trabalho de conclusão de curso (Graduação em Bacharelado em Ciência da Computação) –Universidade Estadual do Oeste do Paraná – UNIOESTE, Cascavel, PA. Disponível em:<https://www.inf.unioeste.br/sitecomp/tcc/2012/TCC_Bruno.pdf>. Acesso em: 13 de Fevereiro de 2022, às 23h05min.